CS228: Introduction to Deep Learning

Lecture 8: PyTorch (cont.) and optimizers

Instructor: Zhouxing Shi

Assignment 1 clarification

- For this assignment, you are supposed to use basic tensor operations, not high-level implementations (such as torch.nn.Linear)
  - Do not use anything from torch.nn

Tensor in PyTorch

- Tensor is a basic data structure in PyTorch
  - Similar to arrays but with various **tensor operations**
  - GPU accelerated
  - Autodiff

```python
import torch
arr = [1, 2, 3, 4, 5]
tensor = torch.tensor(arr)
```

```python
for i in range(len(arr)):
  arr[i] += 1
print(arr)

tensor += 1
print(tensor)
```

```
[2, 3, 4, 5, 6]
tensor([2, 3, 4, 5, 6])
```

MatMul

- Matrix multiplication for tensors

```
a = torch.randn(3, 5)
b = torch.randn(5, 4)
# built-in tensor matmul with acceleration
c_matmul = torch.matmul(a, b)

# equivalent brute-force implementation
c_matmul_bf = torch.zeros(3, 4)
for i in range(a.shape[0]):
  for j in range(b.shape[1]):
    for k in range(a.shape[1]):
      c_matmul_bf[i][j] += a[i][k] * b[k][j]
```

MatMul

- Batched matmul:

```
a = torch.randn(10, 3, 5) # Batched with batch_size = 10
b = torch.randn(5, 4)
# built-in tensor matmul with acceleration
c_matmul = torch.matmul(a, b)

# equivalent brute-force implementation
c_matmul_bf = torch.zeros(10, 3, 4)
for t in range(a.shape[0]):
  for i in range(a.shape[1]):
    for j in range(b.shape[1]):
      for k in range(a.shape[2]):
        c_matmul_bf[t][i][j] += a[t][i][k] * b[k][j]
```

Elementwise nonlinear functions

- torch.exp
- torch.log
- torch.sqrt
- torch.sin
- …
- Similar to math.exp, math.log, math.sqrt, math.sin, …

Neural Network

- Each neural network model or module is implemented by inheriting “torch.nn.Module”
- Implement “__init__” and “forward” functions

```python
class ReLU(nn.Module):
    """ReLU activation function."""

    def __init__(self):
        super().__init__()

    def forward(self, input: torch.Tensor) -> torch.Tensor:
        ####################
        # TODO: apply the ReLU activation
        output = None
        ####################
        return output
```

```python
# Create a module
module = ReLU()

# Use the module
output = module(z)
```

Neural Network

- Leverage existing components:
  - nn.Linear
  - nn.ReLU, nn.Sigmoid
  - nn.CrossEntropyLoss

Training pipeline

- Create the model and optimizer
- Load the dataset and create data loaders with a *batch size*
- Epochs: each *epoch* goes over the training dataset once
- For each iteration in an epoch:
  - Get a batch of data
  - Forward pass to get outputs
  - Calculate the loss function on the outputs
  - Backward pass to get gradients
  - Call the optimizer to update parameters

PyTorch Resources

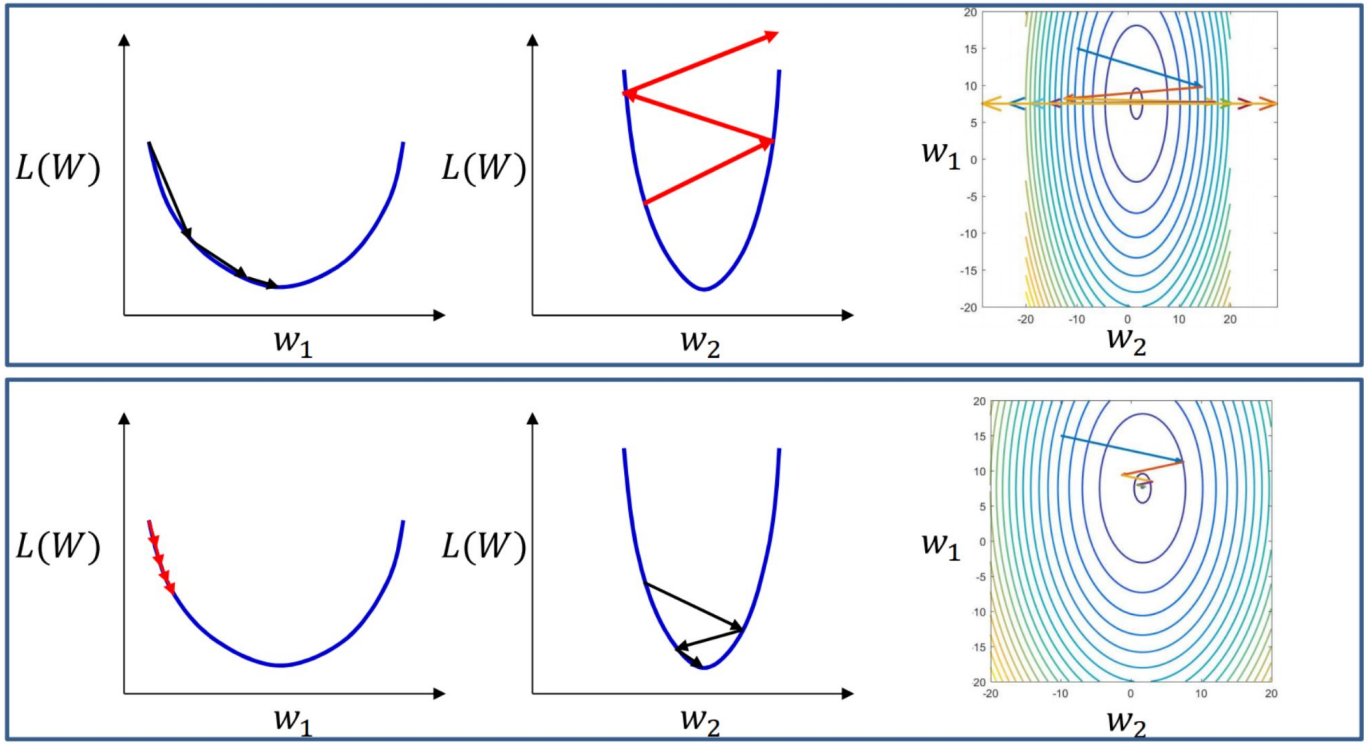
- PyTorch tutorials: https://pytorch.org/tutorials/
- PyTorch documentation: https://pytorch.org/docs/stable/index.html

Limitation of Plain SGD

- A large number of parameters are optimized
- Plain SGD applies **a single learning rate to all the parameters**

$$\mathbf{W}^{(t+1)} \leftarrow \mathbf{W}^{(t)} - \alpha \nabla_{\mathbf{W}^{(t)}} f(\mathbf{W}^{(t)})$$

Limitation of Plain SGD



It converges in some directions but oscillate or diverge in others

The fix

- Use different “effective” learning rates for different directions (parameters)
  - Use larger learning rates for directions that converge smoothly
  - Use smaller learning rates for directions that bounce around

Momentum

- Track oscillations: maintain an average of gradients
  - Large for directions with smooth convergence
  - Small for directions with swings (positive and negative gradients cancelling out)

Momentum

- Update parameters using the *exponential moving average* rather than the current gradients

Plain SGD $$\mathbf{W}^{(t+1)} \leftarrow \mathbf{W}^{(t)} - \alpha \nabla_{\mathbf{W}^{(t)}} f(\mathbf{W}^{(t)})$$

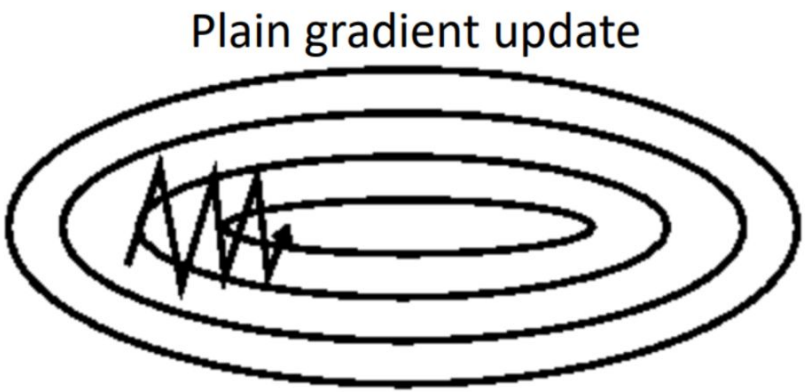
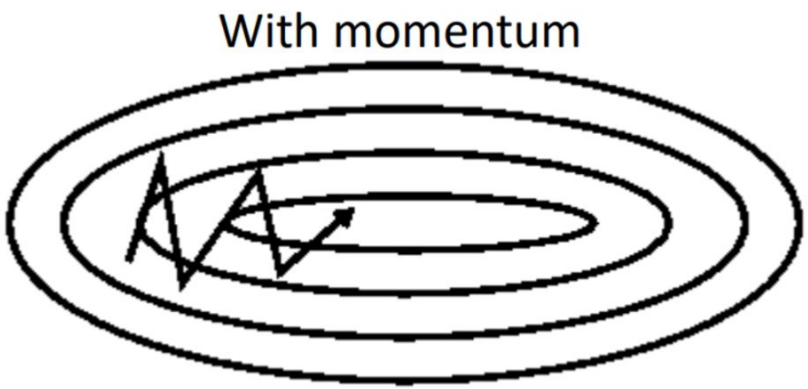
SGD with momentum e.g., β=0.9

$$\Delta \mathbf{W}^{(t+1)} = \beta \Delta \mathbf{W}^{(t)} - \alpha \nabla_{\mathbf{W}^{(t)}} f(\mathbf{W}^{(t)})$$

$$\mathbf{W}^{(t+1)} \leftarrow \mathbf{W}^{(t)} + \Delta \mathbf{W}^{(t+1)}$$

Exponential moving average (geometrically weighted sum):

$$\Delta \mathbf{W}^{(t+1)} = -\alpha \sum_{i=1}^{t} \beta^{t-i} \nabla_{\mathbf{W}^{(i)}} f(\mathbf{W}^{(i)})$$

Momentum

- Larger step size where gradients stay in the same sign
- Smaller step size where the sign keeps flipping